Tentamen TEN1 HI1029 2024-06-04

Sista ordinarie insläpp: 8.00 – dörren stängs.
Tentavakterna delar ut konton. Logga in i tentasystemet så fort som möjligt.
När alla loggat in (förhoppningsvis 8.15) börjar tentamen och pågår i 4h 45min (tex 8.15-13.00).
8.30 Insläpp för försenade.

Extra skrivtid 50%: 7h 10min (tex 8.15-15.25) *(endast 1h extra skrivtid för A eller B)*
Skrivvakter: Anteckna start- och inlämningstid. Jour: Nicklas Brandefelt 087909738

Hjälpmedel: papper (tomma), penna, mus

Logga in med tentamenskontot ni får av skrivvakten. Logga inte ut förrän ni är klara med hela tentan.

Skapa ett nytt projekt i IntelliJ: **Project Name:** TEN1, **Project Location:** H

Skapa ett package per uppgift och kalla dessa uppgift1o2, uppgift3, uppgift4, uppgift 5, uppgift6. Kopiera hjälpfiler från P/HI1029/ till rätt package.

Svaren på teorifrågorna (fråga ett och två) skall besvaras i en textfil döpt till teori.txt som ni lägger i package uppgift1o2.

Även om ni inte löser hela uppgiften så kan ni få delpoäng men ni måste se till att koden kompilerar och kan köras. Både output och koden bedöms och det är viktigt att denna är välskriven och uppfyller uppgiftens instruktioner. Om ett program är ineffektivt kan detta ge poängavdrag. Det är meningen att man skall använda de tekniker vi lärt oss där detta är fördelaktigt. Välj bra namn för variabler, metoder och klasser. Se till att programmet har en bra struktur. Ni behöver på denna tentamen inte kommentera.

Hjälpfiler hittar ni i P/HI1029/ och Java API i P/Java/api/index.html.

Del 1 (15p): För godkänt krävs minst 1 poäng på teorifrågorna samt 9 poäng totalt. Om man får minst en poäng på teorifrågorna får man betyg enligt: 9p – E, 10p – D, 12p – C. Om man inte får minst en poäng på teorifrågorna men minst 8,5 poäng totalt får man Fx.

Del2: Bedöms bara om man får betyg C på del 1 och bedöms då med A, B eller C. Observera att om du har extra skrivtid får du bara använda 1 extra timme för att kunna få betyg A eller B.

Examinator och rättande lärare: Nicklas Brandefelt

Del 1

1. Vad är en abstrakt datatyp? Hur förhåller sig en abstrakta datatyper till datastrukturer? Ge exempel på en abstrakt datatyp och beskriv denna kort och förhållandet till datastrukturen. Observera att vi här menar abstrakta datatyper så som de definieras i vår kurs och i litteraturen i allmänhet. Inte så som de definieras i vår bok i C-programmering. (1p) *Svaret skall lämnas i en textfil, teori.txt i package uppgift1o2.*

2. Ange ordo för antal jämförelser vid urvalssortering (selection sort). Motivera ditt svar tydligt. Ange ordo för antal platsbyten vid urvalssortering. Motivera även detta tydligt. (1p) *Svaret skall lämnas i en textfil, teori.txt i package uppgift1o2.*

3. I katalogen: P/HI1029/uppgift3 hittar du filen SingleLinkedList.java. Där finns också filen Main.java som ska fungera att köra när du är klar med respektive uppgift.

   a) Skriv klart listan så att iteratorn fungerar som det är tänkt och main skriver ut alla element med hjälp av iteratorn. Du behöver inte implementera remove. (2p)

   b) Lägg till en metod iterator(index) i listan där man kan välja vilket element iteratorn skall börja med. Obs att den skall iterera endast till sista elementet så att om man börjar på element nummer 2 kommer den aldrig att nå element 0 och 1. (1p)

4. I denna uppgift ska vi hitta en väg från övre vänstra hörnet till nedre högra hörnet på en matris som är sådan att vi aldrig besöker en plats som innehåller ett heltal som vi tidigare besökt. Matrisen består av positiva heltal under 1000. Vi förflyttar oss genom att gå ett steg åt höger, nedåt eller åt vänster (förutsatt att vi inte hamnar utanför matrisen).

   Skriv en metod som tar en heltalsmatris som in-parameter och returnerar true om det finns en väg och false om det inte gör det. För full poäng ska den använda en bra backtracking algoritm för att lösa problemet (2 poäng kan man få även för andra lösningar). Ett skal hittar du i P/HI1029/uppgift4. Det är ok om metoden förstör matrisen under arbetet.(4p)

   I exemplet nedan har en korrekt väg markerats (finns fler).

31	32	33	34	37	11	36
32	33	35	26	35	36	39
31	32	13	32	31	37	37
11	39	13	14	15	16	17
18	19	33	32	31	32	33
16	38	21	22	23	24	25

5. Implementera en min-heap för heltal. Denna skall fungera med skalet som du hittar i P/HI1029/uppgift5. Implementera heapen med en fast max-storlek på 1000. Du får inte använda några klasser i JCF. Metoden offer ska returnera true om den lyckas och false om den misslyckas. Metoden poll ska returnera Integer.MAX_VALUE om heapen är tom. Du behöver inte implementera några fler metoder. (4p)

   Tips: En heap kan implementeras med en array som inre datastruktur. En nod med index p har ett vänster-barn med index 2p+1, ett höger-barn med index 2p+2 och förälder med index (p-1)/2.

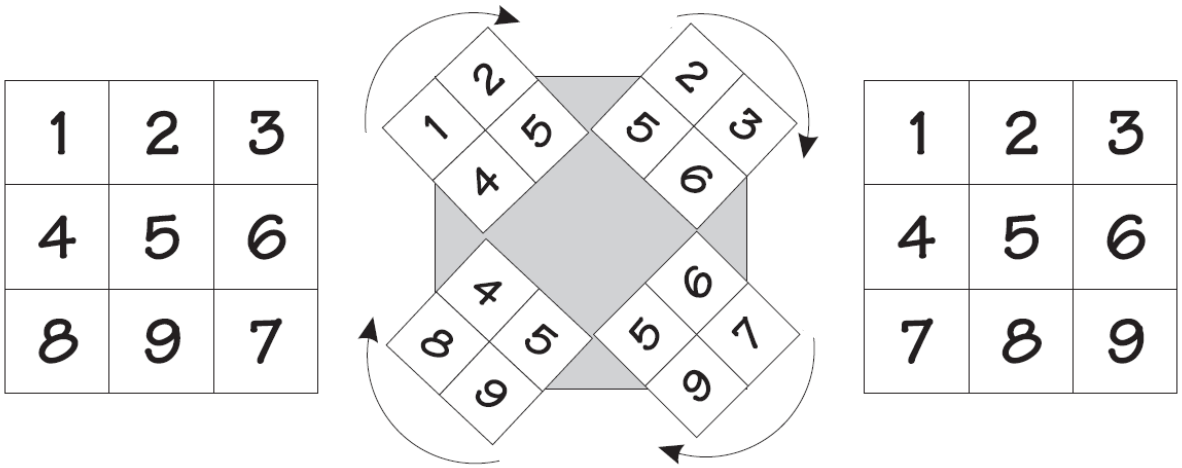
   Algoritm för insättning:
   1. Sätt in nytt heltal på första lediga plats
   2. Medans heltalet inte är i rotnoden och är mindre än förälderns heltal:
      byt plats på dessa

   Algoritm för uttagning:
   1. Ersätt heltalet i rotnoden med heltalet i sista noden i trädet (vi kallar elementet S)
   2. Så länge S har barn och S är större än något av sina barn:
      byt plats på S och minsta barnet

6. I den här uppgiften utgår vi ifrån vårt binära sökträd från föreläsningen (förenklat). Utgå ifrån filerna i P/HI1029/uppgift6. Skriv metoden complete som returnerar true om trädet är komplett annars false. Ett tomt träd är komplett. För full poäng ska den vara så effektiv som möjligt. Kom ihåg att metoden ska kunna hantera även ett tomt träd. Du får inte lägga till några medlemsvariabler men får använda privata hjälpmetoder. (2p)

Del 2

7. I följande pussel ska man få siffrorna i nummerordning genom att rotera de fyra siffrorna i övre vänstra hörnet, övre högra hörnet, nedre vänstra hörnet eller nedre högra hörnet enligt bilden:



Varje 2×2 ruta kan roteras medurs 90 grader. Ett drag består av en sådan rotation.

Skriv en metod som tar en 3×3-matris och returnerar antalet drag som behövs för att lösa den. Om lösning saknas ska den returnera -1. Den ska inte behöva använda något maxdjup och försök effektivisera den så bra som möjligt. Ett skal finns i P/HI1029/uppgift7.

Exempel:

1 2 3 4 5 6 8 9 7	1 2 3 4 5 6 9 8 7	1 2 3 4 9 8 7 6 5	1 2 3 4 5 7 6 9 8	4 9 1 8 3 7 2 5 6
10 drag	9 drag	2 drag	6 drag	11 drag